

Practically Groovy: Of MOPs and mini-languages

Groovy moves the Meta Object Protocol out of the lab and into your apps

Skill Level: Intermediate

[Andrew Glover](#) (andrew@thirstyhead.com)

Co-Founder

ThirstyHead.com

[Scott Davis](#) (scott@thirstyhead.com)

Founder

ThirstyHead.com

20 Sep 2005

Put your ear to the ground and listen closely -- MOP is on the move! Get a primer on the Meta Object Protocol, an old-is-new approach to building applications, languages, and applications as languages.

In a recent interview, Groovy Project Manager Guillaume Laforge mentioned that his favorite Groovy feature is its implementation of the *Meta Object Protocol*, or MOP. This protocol enables an object to make specific choices that affect its own state or behavior when methods, or *messages*, are passed to it at runtime. As described by the PARC Software Design Area projects homepage (see [Resources](#)):

The metaobject protocol approach ... is based on the idea that one can and should open languages up, allowing users to adjust the design and implementation to suit their particular needs. In other words, users are encouraged to participate in the language design process.

Clearly, this broad-mindedness suggests some exciting possibilities for building smarter applications, and even *languages*. In this month's column, I'll show you how Groovy implements MOP, and then use a working example to introduce you to one of its most exciting practical applications: a dictionary app that functions as a

mini-language!

You'll find the source code for the dictionary application example in the [Download](#) section. See [Resources](#) to download DbUnit, which you will need to follow the example.

About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting (or *dynamic*) languages can be an extremely powerful addition to your toolkit, but only when applied properly to appropriate scenarios. To that end, *Practically Groovy* is a series of articles dedicated to exploring the practical uses of Groovy, and teaching you when and how to apply them successfully.

The magical MOP

The Meta Object Protocol is not unique to Groovy, nor was it invented by Groovy's makers. In fact, it can trace its lineage to LISP and to some of the people behind AOP. Given that lineage, it shouldn't be a surprise to find MOP embraced by the cosmopolitan creators of Groovy.

A MOP implementation is possible in Groovy because every object in *Groovyland* implicitly implements `groovy.lang.GroovyObject`, which defines two methods dubbed `invokeMethod()` and `getProperty()`. At runtime, if a message is passed to an object that doesn't exist as a property or method defined in the class or its hierarchy, `getProperty()` or `invokeMethod()` is called.

In Listing 1, I've defined the Groovy class `MOPHandler`, which implements `invokeMethod()` and `getProperty()`. Once I've created an instance of `MOPHandler`, I can call any number of methods or properties and watch as it prints out messages explaining what was invoked.

Listing 1. MOP gets a handle on invocation

```
class MOPHandler {
    def invokeMethod(String method, Object params) {
        println "MOPHandler was asked to invoke ${method}"
        if(params != null){
            params.each{ println "\twith parameter ${it}" }
        }
    }

    def getProperty(String property){
        println "MOPHandler was asked for property ${property}"
    }
}

def hndler = new MOPHandler()
```

```
hndler.helloWorld()
hndler.createUser("Joe", 18, new Date())
hndler.name
```

Go ahead, run the code from Listing 1 and you'll see the output shown in Listing 2.

Listing 2. You didn't believe me, did you?

```
aglover@glove-ubuntu:~/projects/groovy-mop$ groovy
./src/groovy/com/vanward/groovy/MOPHandler1.groovy
MOPHandler was asked to invoke helloWorld
MOPHandler was asked to invoke createUser
    with parameter Joe
    with parameter 18
    with parameter Sun Sep 04 10:32:22 EDT 2005
MOPHandler was asked for property name
```

Pretty slick, isn't it? MOP works well as a safety net to catch message passing errors, but that's not its most interesting function. The virtues of MOP come to light when you use it to create smart objects that can intelligently respond in a *generic fashion* to any message passed to them.

Make me a mini-language

One of the interesting things you can do with MOP is create pseudo-domain-specific languages, or *mini-languages*. These are unique languages targeted at solving specific problems. Unlike prevailing languages such as Java, C#, or even Groovy, which are considered generic languages applicable for any problem, mini-languages fill niches. Need an example? Consider Unix with its venerable shells like Bash.

Just for fun -- and so you can really get a *handle* on MOP -- I'll spend the rest of this column creating a dictionary application that is actually its own mini-language. The application will provide an interface for querying a dictionary. It will allow users to create new word entries, obtain the definition of a given word, obtain synonyms of a given word, query for a word's part of speech, and remove words. Table 1 summarizes this mini-language.

Table 1. Semantics of the dictionary mini-language

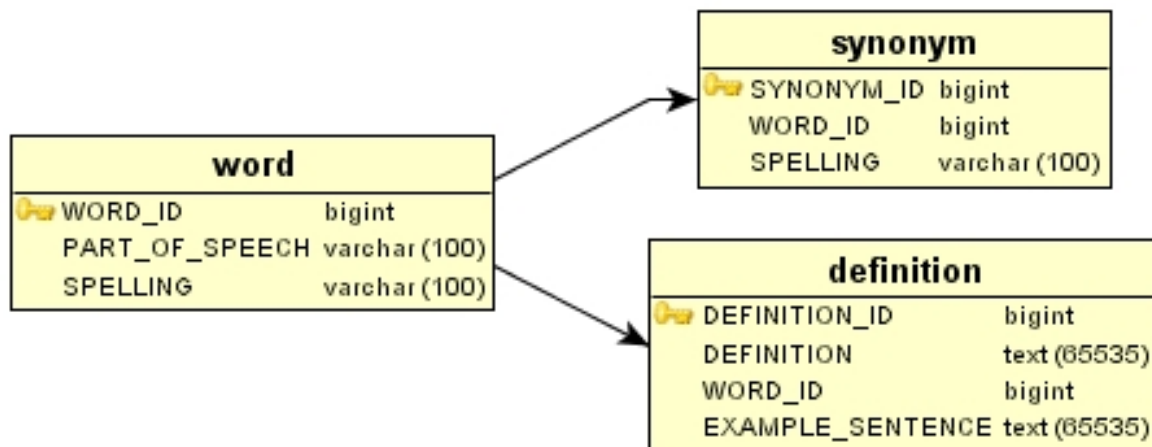
The dictionary application language has the following semantics (listed from most specific to generic):	
1. Parts of speech	To query a word's part of speech, messages should start with <i>is</i> , followed by the word, then a <i>or</i> and then the part of speech.
2. Synonyms	To query a word's list of synonyms, messages should start with <i>synonymsOf</i> , followed by the word.
3. Removing words	To remove a word from the dictionary, messages

	can start with <code>remove</code> or <code>delete</code> , followed by the word.
4. Creating words	To query the definition of a word, pass the word as a property or a method.
5. Retrieving a definition	To create a new word in the dictionary, pass the word as a method and its definition, part of speech, and an optional list of synonyms as parameters.

The dictionary application

The dictionary application will sit on a database with the table structure shown in Figure 1. If you're a regular reader, you may recognize the table structure from the column "[Mark it up with Groovy Builders](#)."

Figure 1. Simple database model for a dictionary



Note: I'm going to ignore definition's EXAMPLE_SENTENCE column.

I'll create a simple facade that will serve as the interface between the user and the database. The facade will utilize Groovy's MOP features by providing implementations of `invokeMethod` and `getProperty`. The `invokeMethod` method will determine the command and delegate responsibility to the corresponding internal private method.

Some prerequisites

Because the application will rely on a database, you may want to refresh your memory of [GroovySql](#) before going further. I'll also be using Groovy's regular expressions (introduced in [Feeling Groovy](#)) to determine the messages passed to the facade.

I'll test the facade *with* Groovy as I write it, so you may want to review Groovy's unit testing abilities, which I covered in "[Unit test your Java code faster with Groovy](#)."

Finally, I'll use DbUnit to manage the state of the database throughout testing. Since I've never written about DbUnit in this series, I'll give you a little primer on using DbUnit with Groovy before going on.

DbUnit, meet Groovy

DbUnit is a JUnit extension that places your database into a known state between test runs. Using DbUnit within Groovy is amazingly simple because DbUnit offers an API that allows for delegation within a test case. To use DbUnit, I need to provide it with a database connection and a file containing the data I'd like the database seeded with. Once I plug these into JUnit's fixture mechanism (that is, `setUp()`), I'm good to go! Listing 3 shows the dictionary application's beginning test class.

Listing 3. Dictionary application's beginning test class

```
package test.com.vanward.groovy

import com.vanward.groovy.SimpleDictionary
import groovy.util.GroovyTestCase
import java.io.File
import java.sql.Connection
import java.sql.DriverManager
import org.dbunit.database.DatabaseConnection
import org.dbunit.database.IDatabaseConnection
import org.dbunit.dataset.IDataset
import org.dbunit.dataset.xml.FlatXmlDataSet
import org.dbunit.operation.DatabaseOperation

class DictionaryTest extends GroovyTestCase{
    def dictionary

    void setUp() {
        this.handleSetUpOperation()
        dictionary = new SimpleDictionary()
    }

    def handleSetUpOperation() {
        def conn = this.getConnection()
        def data = this.getDataSet()
        try{
            DatabaseOperation.CLEAN_INSERT.execute(conn, data)
        }finally{
            conn.close()
        }
    }

    def getDataSet() {
        return new FlatXmlDataSet(new File("test/conf/words-seed.xml"))
    }

    def getConnection() {
        Class.forName("com.mysql.jdbc.Driver")
        def jdbcConnection = DriverManager.
            getConnection("jdbc:mysql://localhost/words",
```

```

        "words", "words")
    return new DatabaseConnection(jdbcConnection)
}
}

```

In Listing 3 I've created the class shell that will serve as my test class as I write the dictionary application. When a test is invoked, JUnit will call `setUp()`, which will then call DbUnit's API. DbUnit will insert the data found in the file `test/conf/words-seed.xml` into the database. The contents of the file `test/conf/words-seed.xml` can be seen in Listing 4.

Listing 4. Sample seed file

```

<?xml version='1.0' encoding='UTF-8'?>
<dataset>
  <word WORD_ID="1" SPELLING="pugnacious" PART_OF_SPEECH="Adjective"/>
  <definition DEFINITION_ID="10"
    DEFINITION="Combative in nature; belligerent."
    WORD_ID="1" />
  <synonym SYNONYM_ID="20" WORD_ID="1" SPELLING="belligerent"/>
  <synonym SYNONYM_ID="21" WORD_ID="1" SPELLING="aggressive"/>
  <word WORD_ID="2" SPELLING="glib" PART_OF_SPEECH="Adjective"/>
  <definition DEFINITION_ID="11"
    DEFINITION="Performed with a natural, offhand ease"
    WORD_ID="2" />
  <definition DEFINITION_ID="12"
    DEFINITION="Marked by ease and fluency of speech or
    writing that often suggests or stems from insincerity,
    superficiality, or deceitfulness"
    WORD_ID="2" />
  <synonym SYNONYM_ID="30" WORD_ID="2" SPELLING="artful"/>
  <synonym SYNONYM_ID="31" WORD_ID="2" SPELLING="suave"/>
  <synonym SYNONYM_ID="32" WORD_ID="2" SPELLING="insincere"/>
  <synonym SYNONYM_ID="33" WORD_ID="2" SPELLING="urbane"/>
  <word WORD_ID="3" SPELLING="rowel" PART_OF_SPEECH="Verb"/>
  <definition DEFINITION_ID="50"
    DEFINITION="to vec, trouble"
    WORD_ID="13" />
</dataset>

```

The seed file's XML elements (shown in Listing 4) match table names. The element's attributes match corresponding table columns.

Building a mini-language

Now that I have my test class defined I can start developing (and testing) the application. I'll tackle each feature in the order listed in [Table 1](#).

Regular expression groups

Regular expressions groups will have a major role to play in the dictionary example. In Groovy, you can create a normal-Java `Matcher` instance via the `=~` syntax. With a `Matcher` instance, you can obtain `String` snippets by invoking the `group()` method. To

create groups within a regular expression, use parentheses. For example, the regular expression `(synonymsOf)(.*)` creates two groups. One group matches the exact String `"synonymsOf"` and the other matches *any* characters following `"synonymsOf"`. Keep in mind, however, that you must invoke `Matcher's matches()` method first before attempting to obtain a group value.

1. Parts of speech

If a user wants to query the part of speech for a word, he or she can pass in a message that looks something like `isRoweIAVerb`, or `isEstivalAnAdjective`. (Note that I'm following camel-case semantics and trying to obey proper English by allowing both a and an.) The intelligent logic for answering the question becomes a matter of determining the proper word, which in the first case is *Rowe* and in the second is *Estival*. The part of speech has to be determined too, which in the examples would be *Verb* and *Adjective*, respectively.

The logic becomes trivial when you put regular expressions into action. The pattern becomes `is(.*)(An|A)(Verb|Adjective|Adverb|Noun)`. I'm only interested in the first and third groups (the word and the part of speech). Given those, I can write a simple database query that retrieves the part of speech and compares the question with the answer, as shown in Listing 5.

Listing 5. Determining the part of speech

```
private determinePartOfSpeech(question){
  def matcher = question =~ 'is(.*)(An|A)(Verb|Adjective|Adverb|Noun)'
  matcher.matches()
  def word = matcher.group(1)
  def partOfSpeech = matcher.group(3)
  def row = sql.firstRow("select part_of_speech from word
    where spelling=?", [word])
  return row[0] == partOfSpeech
}
```

Listing 5 looks pretty simple, but let's see what happens when I write a few test cases for it.

Listing 6. Testing the determination of a word's part of speech

```
void testPartOfSpeechFalse() {
  def val = dictionary.isPugnaciousAVerb()
  assertFalse("pugnacious is not a verb", val)
}
void testPartOfSpeechTrue() {
  def val = dictionary.isPugnaciousAnAdjective()
  assertTrue("pugnacious is an Adjective", val)
}
```

Look again at the XML in Listing 4. Because I'm relying on DbUnit to put the

database into a known state before any tests are run, I can assume that the word, *pugnacious*, is valid with its part of speech set to *Adjective*. In Listing 6, I add two simple tests to `DictionaryTest` (see [Listing 3](#)) that ensure the logic works correctly.

2. Synonyms

The semantics for querying synonyms has the following pattern: `synonymsOfBloviat` where the desired word (*Bloviat*) follows `synonymsOf`. The regular expression is even easier: `(synonymsOf)(.*)`. Once the desired word has been found, the logic involves a database query, which joins the word and synonym tables. The returned synonyms are added to a `List` and returned as shown in Listing 7.

Listing 7. `getSynonyms` implementation

```
private getSynonyms(question){
  def matcher = question =~ '(synonymsOf)(.*)'
  matcher.matches()
  def word = matcher.group(2).toLowerCase()
  def syns = []
  sql.eachRow("select synonym.spelling from synonym, word " +
    "where synonym.word_id = word.word_id and " +
    "word.spelling = ?", [word]){ arow ->
    syns << arow.spelling
  }
  return syns
}
```

The tests shown in Listing 8 verify that a word defined with synonyms returns them correctly, and also that one without any synonyms (*Rowel*) returns an empty `List`.

Listing 8. Don't forget to test that method!

```
void testSynonymsForWord() {
  def val = dictionary.synonymsOfPugnacious()
  def expect = ["belligerent", "aggressive"]
  assertEquals("should be: " + expect, expect, val)
}
void testNoSynonymsForWord() {
  def val = dictionary.synonymsOfRowel()
  def expect = []
  assertEquals("should be: " + expect, expect, val)
}
```

3. Removing words

The remove logic in Listing 9 is flexible, because I allow the command to be `remove` or `delete` followed by the word. For example, `removeGlib` and `deleteGlib` are both valid commands. The regular expression therefore becomes: `(remove|delete)(.*)` and the logic is an SQL `delete`.

Listing 9. Delete or remove a word

```
private removeWord(word){
  def matcher = word =~ '(remove|delete)(.*)'
  matcher.matches()
  def wordToRemove = matcher.group(2).toLowerCase()
  sql.execute("delete from word where spelling=?", [wordToRemove])
}
```

Of course, being so flexible means that I must write *at least* two test cases for removing a word, as shown in Listing 10. To verify the words are truly gone, I attempt to retrieve their definitions (see [5. Retrieving a word's definition](#) for more information) and confirm that nothing comes back.

Listing 10. Test both cases

```
void testDeleteWord() {
  dictionary.deleteGlib()
  def val = dictionary.glib()
  def expect = []
  assertEquals("should be: " + expect, expect, val)
}
void testRemoveWord() {
  dictionary.removePugnacious()
  def val = dictionary.pugnacious()
  def expect = []
  assertEquals("should be: " + expect, expect, val)
}
```

4. Creating words

The semantics for creating a new word is a message that doesn't match any query patterns and which contains a parameter list. For example, a message like `echelon("Noun", ["a level within an organization"])` matches this pattern. The word is *echelon* and the first parameter is the part of speech, followed by a List containing definitions and an optional third parameter, which can be a List of synonyms.

As demonstrated in Listing 11, creating a new word in the database becomes a matter of inserting the word and its definition in the proper tables (word and definition); furthermore, if a List of synonyms is present, each one is added.

Listing 11. Creating a word

```
private createWord(word, defsAndSyms){
  def wordId = id++
  def definitionId = wordId + 10
  sql.execute("insert into word
    (word_id, part_of_speech, spelling) values (?, ?, ?)" ,
    [wordId, defsAndSyms[0], word])
  for(definition in defsAndSyms[1]){
```

```

    sql.execute("insert into definition
                (definition_id, definition, word_id) " +
                "values (?, ?, ?)" , [definitionId, definition, wordId])
  }
  //has a list of synonyms has been passed in
  if(defsAndSyms.length > 2){
    def synonyms = defsAndSyms[2]
    synonyms.each{ syn ->
      sql.execute("insert into synonym
                  (synonym_id, word_id, spelling) values (?, ?, ?)" ,
                  [id++, wordId, syn])
    }
  }
}

```

In Listing 11, the primary key logic is dangerously simple; I can boost my confidence with a few tests, though.

Listing 12. Test cases for the create-word logic

```

void testCreateWord() {
  dictionary.bloviat("Verb",
    ["To discourse at length in a pompous or boastful manner"],
    ["orate", "gabble", "lecture"])
  def val = dictionary.bloviat()
  def expect = "To discourse at length in a pompous or boastful manner"
  assertEquals("should be: " + expect, expect, val[0])
}
void testCreateWordNoSynonyms() {
  dictionary.echelon("Noun", ["a level within an organization"])
  def val = dictionary.echelon()
  def expect = "a level within an organization"
  assertEquals("should be: " + expect, expect, val[0])
}

```

As usual, I write a few test cases in Listing 12 to verify things work as expected. After creating a word, I query the dictionary instance for the same word to confirm it made it into the database.

5. Retrieving a word's definition

Any message passed to the dictionary application that doesn't match any of the previous query types (part of speech and synonym) and which doesn't contain any parameters, is assumed to be a definition query. For example, a message like `.glib` or `.glib()` would return *glib*'s definitions.

Listing 13. Finding a word's definitions isn't so hard!

```

private getDefinitions(word){
  def definitions = []
  sql.eachRow("select definition.definition from definition, word " +
    "where definition.word_id = word.word_id and " +
    "word.spelling = ?", [word]){ arow ->
    definitions << arow.definition
  }
}

```

```
    return definitions
}
```

Because I allow both a method call and a property call to work as a definition query, I must write at least two tests in Listing 14. Just like in Listing 6, I can assume *pugnacious* is already in the database. Isn't DbUnit handy?

Listing 14. Test both cases -- method call and property

```
void testFindWord() {
    def val = dictionary.pugnacious()
    def expect = "Combative in nature; belligerent."
    assertEquals("should be: " + expect, expect, val[0])
}
void testFindWordAsProperty() {
    def val = dictionary.pugnacious
    def expect = "Combative in nature; belligerent."
    assertEquals("should be: " + expect, expect, val[0])
}
```

And that's it for the semantics of the dictionary application. Now let's have a deeper look at the logic underlying the application.

The logic of MOP

The meat of the dictionary application lies in my implementations of `invokeMethod` and `getProperty`. In these methods I need to make intelligent decisions on how to proceed when a message is passed to the application. The decisions are found in a series of conditional statements, which execute `String` operations to determine the type of message.

The *only* vexing conditional in Listing 15 happens to be the first one, which attempts to verify if the message is truly a definition query and not one of the other possible combinations such as `is...`, `remove...`, `delete...`, or `synonymOf...`

Listing 15. The logical heart of MOP

```
def invokeMethod(String methodName, Object params) {
    if(isGetDefinitions(methodName, params)){
        return getDefinitions(methodName)
    }else if (params.length > 0){
        createWord(methodName, params)
    }else if(methodName[0..1] == 'is'){
        return determinePartOfSpeech(methodName)
    }else if
        (methodName[0..5] == 'remove' || methodName[0..5] == 'delete'){
        removeWord(methodName)
    }else if (methodName[0..9] == 'synonymsOf'){
        return getSynonyms(methodName)
    }
}
```

```

private isGetDefinitions(methodName, params){
    return !(params.length > 0) &&
        ( (methodName.length() <= 5
          && methodName[0..1] != 'is' ) ||
          (methodName.length() <= 10
          && isRemoveDeleteIs(methodName) ) ||
          (methodName.length() >= 10
          && methodName[0..9] != 'synonymsOf'
          && isRemoveDeleteIs(methodName)))
    }

private isRemoveDeleteIs(methodName){
    return (methodName[0..5] != 'remove'
          && methodName[0..5] != 'delete'
          && methodName[0..1] != 'is')
    }

```

The `isGetDefinitions` method does a fair bit of work and relies on the `isRemoveDeleteIs` to do some of it. Once it's been determined that a message doesn't fit the definition query, the logic becomes much simpler.

The `getProperty` method in Listing 16 is simple since I've stipulated that users can only query for definitions by properties and nothing else; consequently, I call the `getDefinitions` method when `getProperty` is invoked.

Listing 16. Properties are easy, thankfully!

```

def getProperty(String property){
    return getDefinitions(property)
}

```

And that's it! Really. The logic found in Listings 5 through 16 creates an application that bestows a fair bit of flexibility to users with an overall complexity hit that isn't all that bad. As noted previously, of course, the primary key logic found in the application isn't bulletproof. Myriad steps could be taken to improve this aspect, from database sequences to a framework like Hibernate, which handles IDs quite elegantly.

Seeing is believing, so Listing 17 demonstrates the dictionary application's language in full force. (If only I'd had something like this before taking the SATs!)

Listing 17. The Dictionary show

```

import com.vanward.groovy.SimpleDictionary

def dict = new SimpleDictionary()

dict.vanward("Adjective", ["Being on, or towards the front"],
             ["Advanced"])
dict.pulchritude("Noun", ["Great physical beauty and appeal"])

dict.vanward //prints "Being on, or towards the front"

```

```
dict.pulchritude()      //prints "Great physical beauty and appeal"
dict.synonymsOfVanward() //prints "Advanced"
dict.isVanwardANoun()   //prints "false"
dict.removeVanward()
dict.deletePulchritude()
```

The point of MOC

At this point, you may be wondering *what's the point?* I could just have easily exposed those `private` methods, rearranged a few things, and offered an application that behaved normally (that is, I could have exposed a `getDefinition` method, a `createWord` method, etc.) without using MOC.

Let's examine this proposition -- say I want to create the exact same dictionary application without relying on Groovy's MOC implementation. I'd have to redefine the parameter list for the `createWord()` method to look something like `def createWord(word, partOfSpeech, defs, syns=[]){}`.

So as you can see, my first step would be to drop the `private` modifier and replace it with `def`, which is the same as declaring the method as `public`. Next, I'd define the parameters and make the last one optional (`syns=[]`).

I'd also have to define a `delete` method that would, at minimum, call the `remove` method to support both removes and deletes. The part-of-speech query would also have to be altered. I could add another parameter, so it would possibly look something like `determinePartOfSpeech(word, partOfSpeech)`.

In essence, with the above steps I would have shifted the dictionary application's conceptual complexity from the MOP implementation to my static API. Which begs the question -- what would be the point of doing *that*? Surely you can see that by implementing MOP I've opened up unparalleled flexibility to my application users. The resulting API isn't so much a series of statically defined methods as a flexible *language* in itself!

In conclusion

If you think your head is spinning now, think about this: What if you could build the dictionary application to allow users to create queries at will? For instance, `findAllWordsLikeGlib` or `findAllWordsWithSynonymGlib`, and so on -- these semantics become simply a matter of text-parsing on the MOP side and a gold mine of queries on the user's side!

Now think a little more: What if you could create another mini-language that was a

bit more useful in a business sense -- like a mini-language targeting stock trading? What if you went further and built this application to be console friendly? Rather than writing scripts, users would use Groovy's shell. (Remember what I said about Bash?)

If you think this is kind of alchemic utility is for esoteric LISP guys drooling over Emacs extensions, think again! You need only look across the street at those vociferous Ruby on Rails gurus to know why that platform is so exciting. Peek under the hood and you'll see MOP at work, allowing you to create nifty queries without having to drop down into SQL, if you *follow a naming pattern*. Now who's drooling?

Downloads

Description	Name	Size	Download method
Sample code	j-pg09205.zip	1KB	HTTP

[Information about download methods](#)

Resources

Learn

- ["Fast-track your Web apps with Ruby on Rails"](#) (David Mertz, developerWorks, June 2005): What gives Ruby on Rails its dynamic flexibility? MOP, in part.
- ["Control your test-environment with DbUnit and Anthill"](#) (Philippe Girolami, developerWorks, February 2004): More on test-driven development with DbUnit.
- ["AOP banishes the tight-coupling blues"](#) (Andrew Glover, developerWorks, February 2004): A primer on aspect-oriented programming.
- ["Emacs a top-notch Java IDE? You bet!"](#) (Greg Travis, developerWorks, May 2001): Emacs is one of the most popular development environments in the history of software programming, so why have you never tried it?
- ["Interview with Guillaume Laforge"](#): Andrew Glover talks with Groovy's project manager about the JSR, who's using Groovy, and the Meta Object Protocol.
- [The SDA Projects page](#): A listing of PARC's software design projects, including AspectJ and MOP.
- [The Practically Groovy series page](#): Tips and tricks for Groovy programmers.
- [The Java technology zone](#): Hundreds of articles about every aspect of Java programming.

Get products and technologies

- [The Groovy project page](#): Download Groovy.

Discuss

- [developerWorks blogs](#): Get involved in the developerWorks community!

About the authors

Andrew Glover

Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the [easyb](#) Behavior-Driven Development (BDD) framework and is the co-author of three books: [Continuous Integration](#), [Groovy in Action](#), and [Java Testing Patterns](#). He teaches a wide variety of Groovy-, Grails-, and testing-related classes at [ThirstyHead.com](#). You can keep up with Andy at [thediscoblog.com](#) where he routinely blogs about software development.

Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of ThirstyHead.com, a Groovy and Grails training company. His books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#) and [Practically Groovy](#).